UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación

TRABAJO PRÁCTICO INTEGRADOR

Sistema de Gestión de Inventario para una Ferretería

Materia: Programación I

Estudiante: Lucas Córdoba

Año académico: 2026

Lenguaje utilizado: Python

Entorno de desarrollo: Visual Studio Code

Introducción

El presente Trabajo Práctico Integrador tiene como objetivo desarrollar un sistema de gestión de inventario utilizando el lenguaje de programación Python, aplicando los conocimientos adquiridos durante la cursada de Programación I.

El programa está pensado para administrar las herramientas de una ferretería, permitiendo cargar productos, consultar las cantidades disponibles, identificar herramientas agotadas y actualizar el stock mediante operaciones de venta e ingreso de mercadería.

Para su desarrollo se utilizaron diferentes conceptos de programación, como funciones, estructuras condicionales y repetitivas, listas, diccionarios y manejo de errores. Estos elementos permiten organizar el código, validar los datos ingresados por el usuario y evitar operaciones incorrectas.

A lo largo del trabajo se explicará el funcionamiento del sistema, su estructura y las principales decisiones tomadas durante su desarrollo. También se presentarán las pruebas realizadas para comprobar que las distintas funciones respondan correctamente ante situaciones habituales y entradas inválidas.

Este proyecto permitió integrar los contenidos estudiados y ponerlos en práctica mediante la resolución de un problema concreto, fortaleciendo tanto la lógica de programación como la capacidad de identificar y corregir errores.

Objetivos

Objetivo general

Desarrollar un sistema de gestión de inventario para una ferretería utilizando el lenguaje de programación Python, que permita administrar las herramientas y sus cantidades disponibles mediante un menú interactivo, aplicando los conocimientos adquiridos durante la cursada de Programación I.

Objetivos específicos

- Aplicar estructuras condicionales y repetitivas para controlar el funcionamiento del programa y las diferentes opciones del menú.
- Utilizar listas y diccionarios para almacenar y organizar la información de las herramientas y sus cantidades disponibles.
- Implementar funciones independientes para cargar herramientas, mostrar el inventario, consultar el stock, identificar productos agotados y registrar movimientos de mercadería.
- Incorporar validaciones y manejo de excepciones para evitar el ingreso de datos incorrectos, como nombres vacíos, cantidades negativas o valores no numéricos.
- Desarrollar operaciones de venta e ingreso de mercadería que permitan actualizar el stock sin generar cantidades negativas.
- Realizar pruebas de funcionamiento para verificar que el programa responda correctamente tanto en situaciones normales como ante errores de ingreso.
- Fortalecer la lógica de programación y la capacidad de identificar, analizar y corregir errores durante el desarrollo del sistema.

Herramientas y entorno de desarrollo

Para realizar el presente Trabajo Práctico Integrador utilicé el lenguaje de programación **Python**, debido a que permite desarrollar programas de manera sencilla y organizada, utilizando estructuras y funciones que facilitan la resolución de diferentes problemas.

Como entorno de desarrollo utilicé **Visual Studio Code**, un editor de código que me permitió escribir, organizar, ejecutar y corregir el programa durante las distintas etapas del trabajo.

El sistema fue desarrollado mediante una aplicación de consola, donde el usuario puede interactuar con un menú de siete opciones para realizar diferentes operaciones relacionadas con la administración del inventario.

Para almacenar la información de las herramientas utilicé una lista de diccionarios, donde cada herramienta contiene su nombre y la cantidad disponible en stock. Esta estructura permite consultar, agregar y modificar los datos mientras el programa se encuentra en ejecución.

Durante el desarrollo también utilicé la terminal integrada de Visual Studio Code para ejecutar el programa y realizar diferentes pruebas de funcionamiento. Estas pruebas me permitieron detectar errores, corregir validaciones y comprobar que las operaciones de venta e ingreso de mercadería actualizaran correctamente las cantidades disponibles.

El programa fue desarrollado utilizando los recursos básicos del lenguaje Python, sin necesidad de incorporar bibliotecas externas ni bases de datos

Descripción y funcionamiento del sistema

El sistema de inventario fue desarrollado para facilitar la administración de herramientas de una ferretería mediante un programa de consola escrito en Python.

Al iniciar el programa, se presenta un menú principal compuesto por siete opciones, desde el cual el usuario puede seleccionar la operación que desea realizar. Este menú permanece disponible hasta que se selecciona la opción de salir.

El inventario se almacena en una lista de diccionarios, donde cada elemento representa una herramienta y contiene dos datos principales: su nombre y la cantidad disponible. Esta información se mantiene durante la ejecución del programa y se comparte entre las distintas funciones.

Las funcionalidades implementadas son las siguientes:

1. **Cargar herramientas:** permite registrar varias herramientas, indicando sus nombres y cantidades iniciales, evitando nombres duplicados y cantidades negativas.
2. **Mostrar inventario:** presenta todas las herramientas registradas junto con su stock disponible.
3. **Consultar stock:** permite buscar una herramienta por su nombre y conocer la cantidad de unidades existentes.
4. **Reporte de agotados:** identifica y muestra las herramientas que tienen una cantidad igual a cero.
5. **Alta de producto:** permite incorporar una nueva herramienta al inventario sin modificar las que ya están registradas.
6. **Actualizar stock:** permite registrar ventas e ingresos de mercadería. En las ventas se descuentan unidades, siempre que exista stock suficiente, mientras que en los ingresos se suman unidades a la cantidad disponible.
7. **Salir:** finaliza la ejecución del programa.

Para organizar el funcionamiento del sistema, cada operación fue desarrollada mediante una función independiente. Además, se incorporaron validaciones y manejo de excepciones para controlar los datos ingresados y evitar que las operaciones incorrectas afecten al inventario.

De esta manera, el programa permite realizar las principales tareas de administración de herramientas desde una interfaz sencilla, aplicando los conceptos fundamentales de programación trabajados durante la cursada.

Estructura del código y explicación de las funciones

Para desarrollar el sistema de inventario, organicé el programa mediante diferentes funciones, asignando a cada una una tarea específica. Esta organización me permitió trabajar de manera más ordenada, identificar errores y realizar modificaciones sin afectar directamente al resto del programa.

1. Función principal: main()

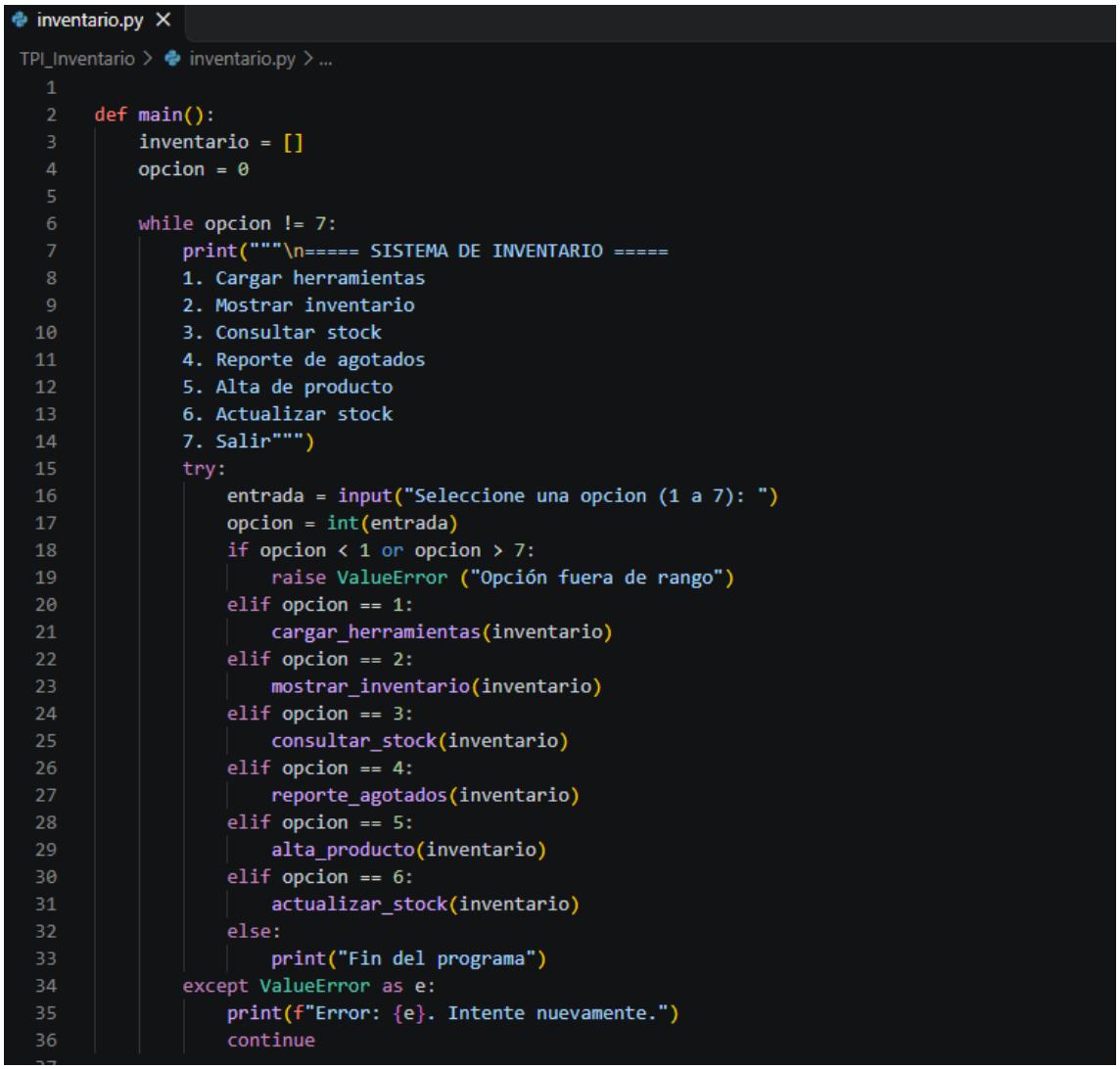
La función main() es la encargada de controlar el funcionamiento general del sistema.

Dentro de esta función se crea la lista inventario, que inicialmente se encuentra vacía y se utiliza para almacenar las herramientas registradas.

También se implementa un ciclo while que permite mostrar continuamente el menú principal hasta que el usuario selecciona la opción 7, correspondiente a salir del programa.

Mediante estructuras condicionales if y elif, el programa identifica la opción seleccionada y ejecuta la función correspondiente.

Además, se utiliza un bloque try-except para controlar errores en los datos ingresados, evitando que el programa finalice inesperadamente cuando el usuario introduce valores incorrectos.



- **Estructura de la función principal del programa.**

En esta captura se observa la función main(), encargada de inicializar el inventario y controlar el menú principal. Mediante un ciclo while y estructuras condicionales, permite ejecutar las distintas funciones del sistema. También incorpora un bloque try-except para controlar los errores ingresados por el usuario.

2. Almacenamiento de datos

Para almacenar la información utilicé una lista de diccionarios.

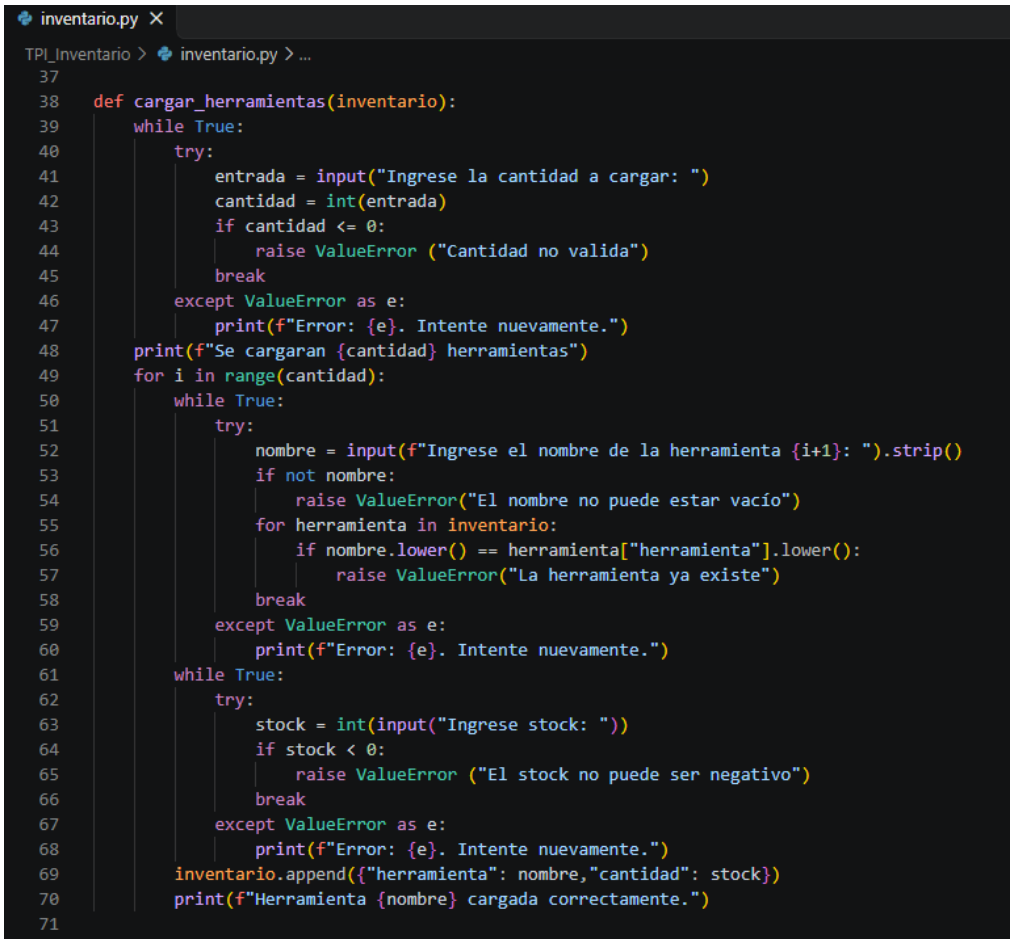
Cada diccionario representa una herramienta y contiene dos claves:

- herramienta: almacena el nombre del producto.
- cantidad: almacena las unidades disponibles.

Por ejemplo:

inventario.append({"herramienta": nombre, "cantidad": stock})

Esta estructura permite agregar herramientas, recorrer el inventario, consultar sus cantidades y modificar el stock sin utilizar una base de datos.




```python
def cargar_herramientas(inventario):
    while True:
        try:
            entrada = input("Ingrese la cantidad a cargar: ")
            cantidad = int(entrada)
            if cantidad <= 0:
                raise ValueError ("Cantidad no valida")
            break
        except ValueError as e:
            print(f"Error: {e}. Intente nuevamente.")
    print(f"Se cargaran {cantidad} herramientas")
    for i in range(cantidad):
        while True:
            try:
                nombre = input(f"Ingrese el nombre de la herramienta {i+1}: ").strip()
                if not nombre:
                    raise ValueError("El nombre no puede estar vacío")
                for herramienta in inventario:
                    if nombre.lower() == herramienta["herramienta"].lower():
                        raise ValueError("La herramienta ya existe")
                break
            except ValueError as e:
                print(f"Error: {e}. Intente nuevamente.")
        while True:
            try:
                stock = int(input("Ingrese stock: "))
                if stock < 0:
                    raise ValueError ("El stock no puede ser negativo")
                break
            except ValueError as e:
                print(f"Error: {e}. Intente nuevamente.")
        inventario.append({"herramienta": nombre,"cantidad": stock})
        print(f"Herramienta {nombre} cargada correctamente.")
```

- **Función de carga de herramientas.**

En esta captura se muestra el procedimiento utilizado para registrar herramientas en el inventario. Se emplean ciclos repetitivos y validaciones para controlar los nombres, evitar productos duplicados y comprobar que las cantidades ingresadas sean válidas. Finalmente, cada herramienta se incorpora a la lista mediante un diccionario.

3. Funciones de gestión del inventario

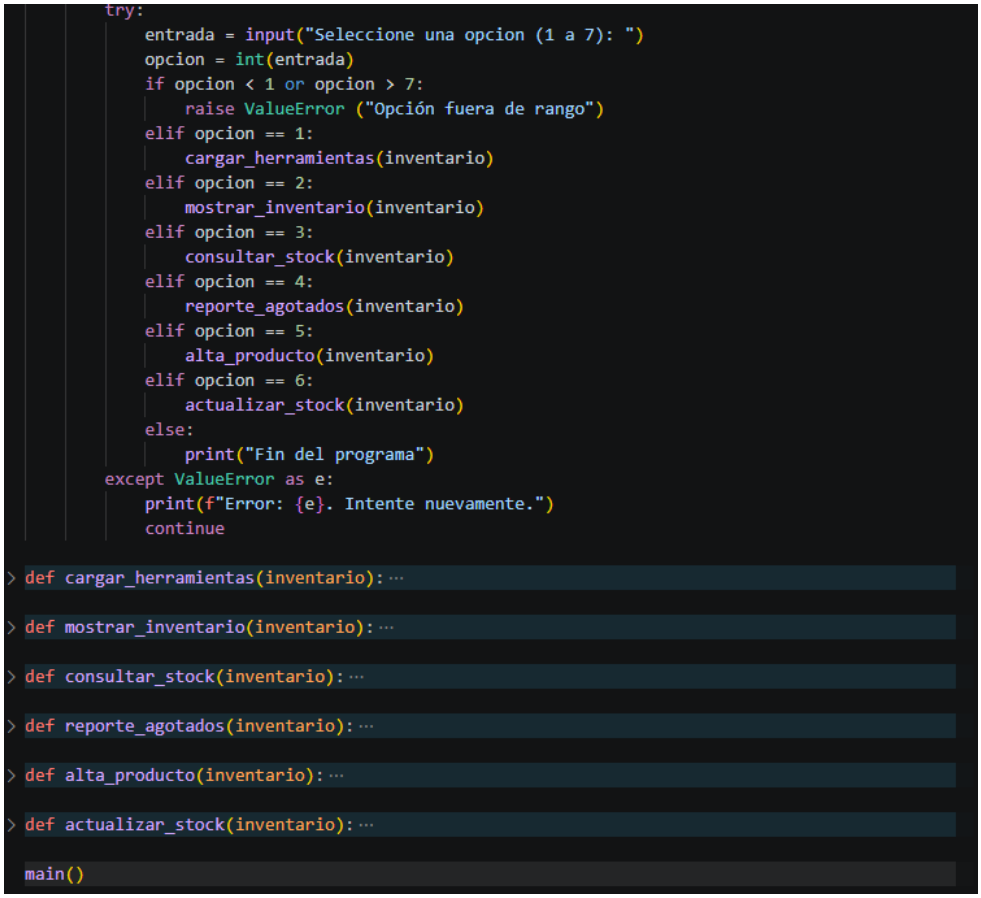
La función cargar_herramientas() permite registrar varias herramientas mediante un ciclo for. Durante la carga se realizan validaciones para evitar nombres vacíos, herramientas duplicadas y cantidades negativas.

La función mostrar_inventario() recorre la lista y muestra las herramientas registradas junto con sus cantidades disponibles. Si el inventario está vacío, informa esta situación al usuario.

La función consultar_stock() permite buscar una herramienta por su nombre. Para realizar la comparación se utiliza el método .lower(), que permite encontrar coincidencias sin distinguir entre mayúsculas y minúsculas.

La función reporte_agotados() identifica las herramientas cuya cantidad es igual a cero. Para ello utiliza un ciclo for, una condición if y una variable booleana llamada encontrado, que permite determinar si existen productos agotados.

La función alta_producto() permite incorporar una herramienta individual al inventario, comprobando previamente que su nombre no esté vacío, que no exista otra herramienta con el mismo nombre y que su cantidad inicial sea válida.



```
        try:
            entrada = input("Seleccione una opcion (1 a 7): ")
            opcion = int(entrada)
            if opcion < 1 or opcion > 7:
                raise ValueError ("Opción fuera de rango")
            elif opcion == 1:
                cargar_herramientas(inventario)
            elif opcion == 2:
                mostrar_inventario(inventario)
            elif opcion == 3:
                consultar_stock(inventario)
            elif opcion == 4:
                reporte_agotados(inventario)
            elif opcion == 5:
                alta_producto(inventario)
            elif opcion == 6:
                actualizar_stock(inventario)
            else:
                print("Fin del programa")
        except ValueError as e:
            print(f"Error: {e}. Intente nuevamente.")
            continue

> def cargar_herramientas(inventario): ···

> def mostrar_inventario(inventario): ···

> def consultar_stock(inventario): ···

> def reporte_agotados(inventario): ···

> def alta_producto(inventario): ···

> def actualizar_stock(inventario): ···

main()
```

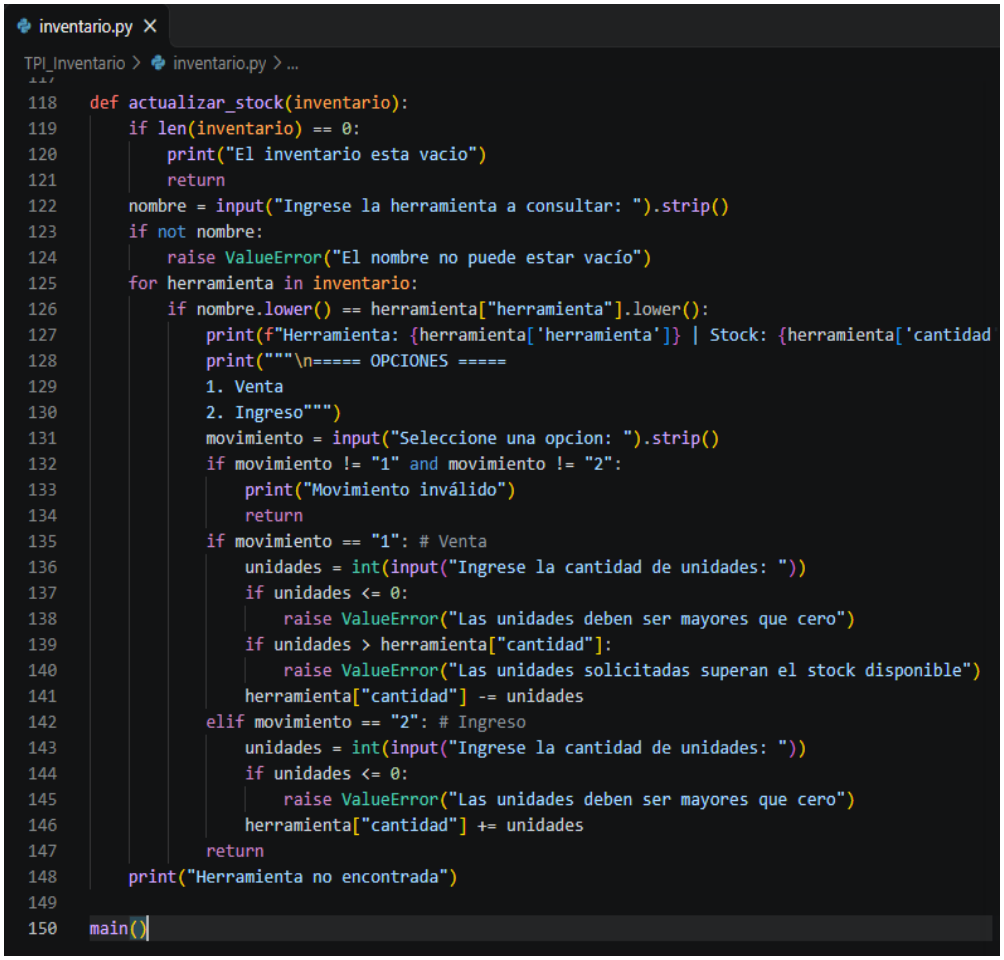
4. Actualización del stock

La función actualizar_stock() permite realizar dos operaciones principales: venta e ingreso de mercadería.

Cuando el usuario selecciona una venta, el programa comprueba que la cantidad solicitada sea mayor que cero y que exista stock suficiente. Si se cumplen estas condiciones, descuenta las unidades mediante el operador -=.

Cuando se selecciona un ingreso de mercadería, el programa valida la cantidad y suma las unidades al stock existente mediante el operador +=.

En ambos casos, las validaciones se realizan antes de modificar el inventario, evitando que una operación incorrecta altere las cantidades almacenadas.



- **Función de actualización de stock.**

En esta captura se observa la función encargada de registrar ventas e ingresos de mercadería. El programa comprueba que las cantidades sean mayores que cero y, en el caso de las ventas, verifica que exista stock suficiente antes de descontar las unidades. Esto permite evitar modificaciones incorrectas en el inventario.

5. Validaciones y manejo de errores

Durante el desarrollo incorporé diferentes controles para evitar errores de funcionamiento.

Entre las principales validaciones se encuentran la detección de nombres vacíos, herramientas duplicadas, cantidades negativas, opciones fuera de rango y ventas superiores al stock disponible.

Para manejar estos casos utilicé la instrucción raise ValueError, que permite generar un error cuando no se cumplen determinadas condiciones, y bloques try-except, que permiten capturarlo y mostrar un mensaje al usuario.

Estas herramientas resultaron importantes para conseguir un programa más seguro frente a entradas incorrectas y facilitar la identificación y corrección de errores durante las pruebas.

Pruebas de funcionamiento y resultados

Una vez finalizado el desarrollo del sistema de inventario, realicé diferentes pruebas desde la terminal integrada de Visual Studio Code para comprobar el funcionamiento de las opciones del menú y verificar que las validaciones respondieran correctamente.

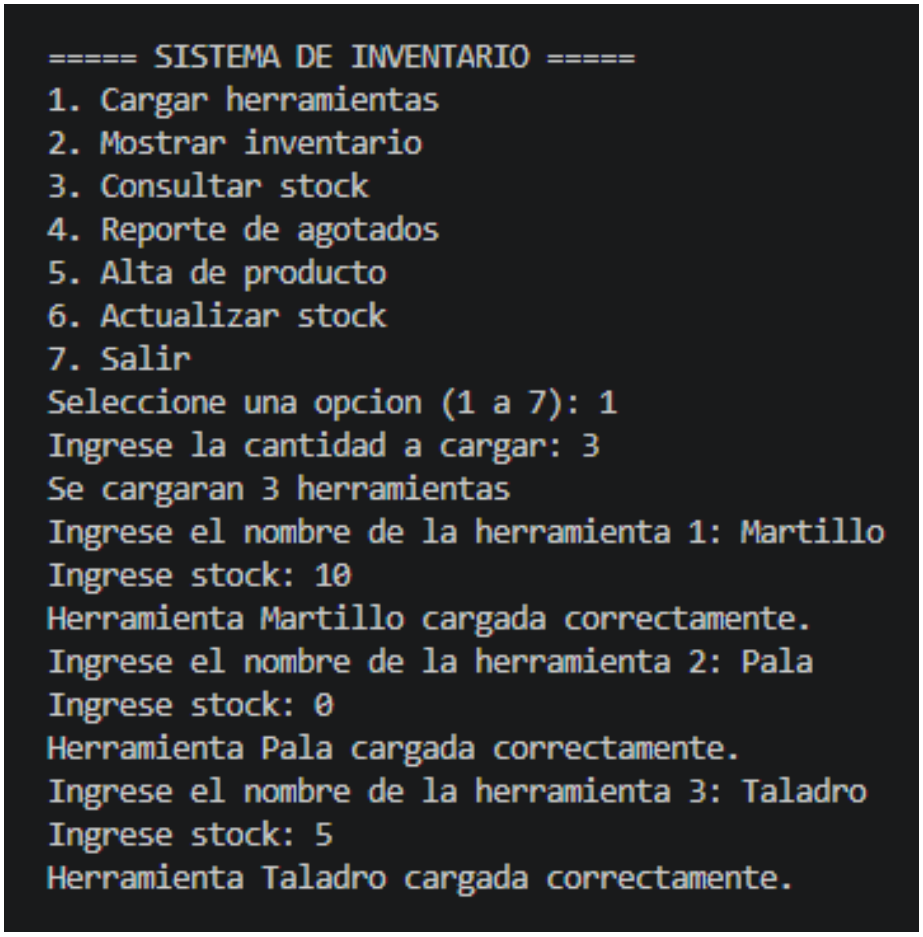
Las pruebas se realizaron utilizando herramientas de ejemplo, como Martillo, Pala y Taladro, con diferentes cantidades disponibles. También ingresé valores incorrectos para comprobar que el programa pudiera detectar errores sin interrumpir su funcionamiento.

Prueba 1. Carga y visualización del inventario

En primer lugar, comprobé que el sistema permitiera registrar herramientas indicando su nombre y cantidad inicial.

Posteriormente, utilicé la opción de mostrar inventario para verificar que los productos cargados aparecieran correctamente junto con sus cantidades disponibles.

Esta prueba permitió comprobar el funcionamiento de la lista de diccionarios utilizada para almacenar la información.



- **Prueba de carga y visualización del inventario.**

En esta prueba realicé la carga inicial de tres herramientas: Martillo, Pala y Taladro, asignándoles cantidades de 10, 0 y 5 unidades respectivamente. Posteriormente, seleccioné la opción 2 del menú para visualizar el inventario y comprobar que los datos se habían almacenado correctamente.

El resultado permitió verificar el funcionamiento de la carga de herramientas y de la lista de diccionarios utilizada para guardar la información. También comprobé que el programa permite registrar herramientas con stock inicial igual a cero.

Prueba 2. Consulta de stock

Utilicé la opción de consultar stock para buscar una herramienta previamente registrada.

El programa mostró correctamente el nombre de la herramienta y la cantidad disponible. También comprobé que fuera posible realizar búsquedas sin distinguir entre mayúsculas y minúsculas.

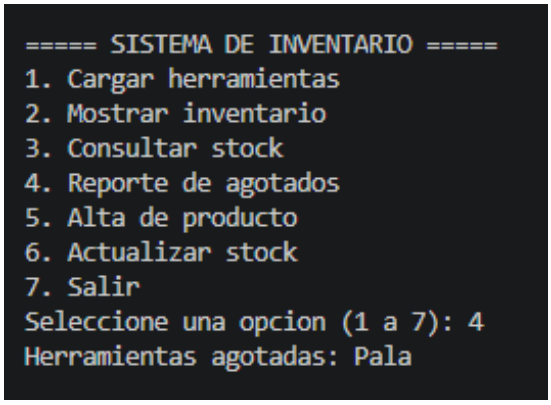
Además, verifiqué que el sistema informara cuando una herramienta no existía o cuando el inventario se encontraba vacío

Prueba 3. Reporte de herramientas agotadas

Para comprobar esta función, utilicé herramientas con diferentes cantidades, incluyendo productos con stock igual a cero.

Al ejecutar el reporte, el programa mostró únicamente las herramientas agotadas.

También comprobé que distinguiera entre un inventario vacío y otro que contenía herramientas, pero ninguna agotada.



- **Prueba del reporte de herramientas agotadas.**

En esta prueba utilicé la opción 4 del menú para comprobar el funcionamiento del reporte de herramientas agotadas. El inventario contenía tres productos: Martillo con 10 unidades, Pala con 0 y Taladro con 5.

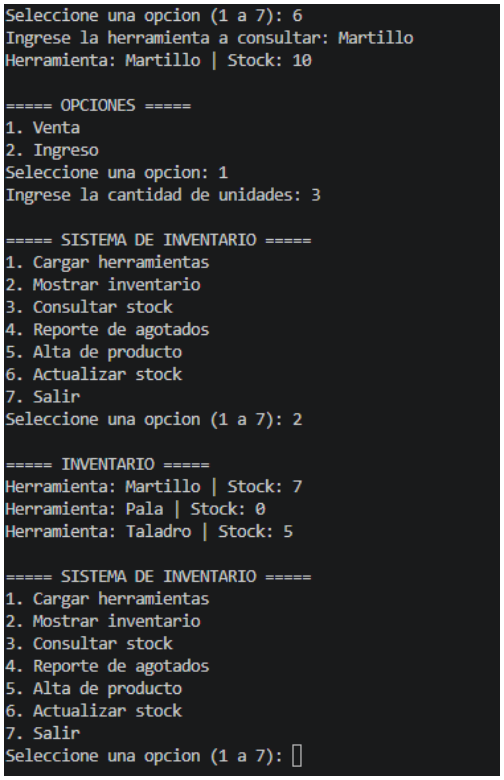
Al ejecutar la función, el programa identificó correctamente a Pala como la única herramienta sin stock disponible. Esto permitió verificar que el sistema recorre el inventario y muestra solamente los productos cuya cantidad es igual a cero.

Prueba 4. Venta de mercadería

Realicé una prueba utilizando una herramienta con 10 unidades disponibles y registré una venta de 3 unidades.

Posteriormente, consulté nuevamente el inventario y comprobé que el stock se había actualizado correctamente a 7 unidades.

Esta prueba permitió verificar el funcionamiento de la operación de resta utilizada para registrar las ventas.



- **Prueba de actualización de stock mediante una venta.**

En esta prueba utilicé la opción 6 del menú para registrar una venta de 3 unidades de la herramienta Martillo, que inicialmente tenía un stock de 10 unidades.

Luego de realizar la operación, seleccioné la opción 2 para visualizar nuevamente el inventario y comprobar que la cantidad disponible se había reducido correctamente a 7 unidades.

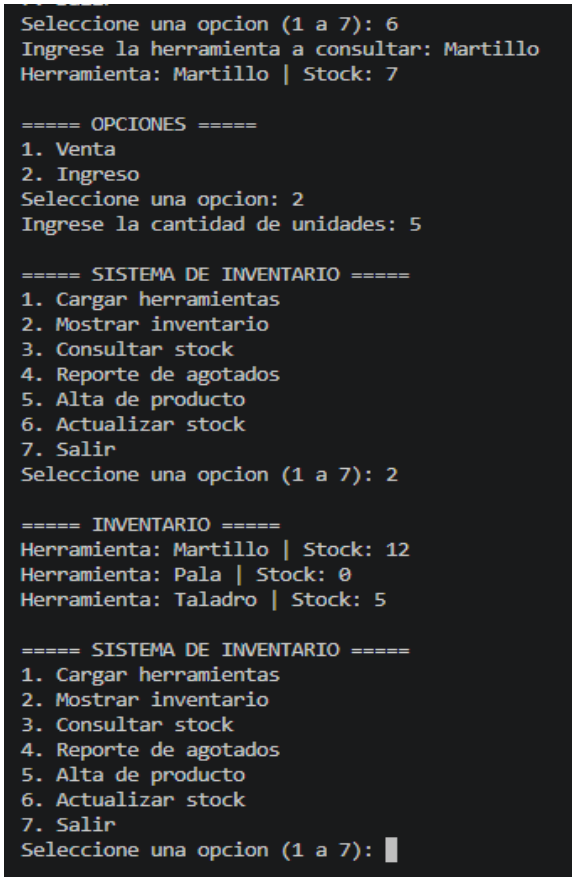
El resultado permitió verificar el funcionamiento de la operación de venta y comprobar que la modificación afectó únicamente a la herramienta seleccionada, manteniendo sin cambios las cantidades de los demás productos.

Prueba 5. Ingreso de mercadería

A partir de las 7 unidades disponibles de la prueba anterior, registré un ingreso de 5 unidades.

Al consultar nuevamente el inventario, comprobé que la cantidad había aumentado a 12 unidades.

De esta manera, verifiqué que el sistema permitiera sumar mercadería al stock existente.



- **Prueba de actualización de stock mediante ingreso de mercadería.**

En esta prueba utilicé la opción 6 del menú para registrar un ingreso de 5 unidades de la herramienta Martillo, que anteriormente contaba con un stock de 7 unidades.

Luego de realizar la operación, seleccioné la opción 2 para consultar nuevamente el inventario y comprobar que la cantidad disponible había aumentado correctamente a 12 unidades.

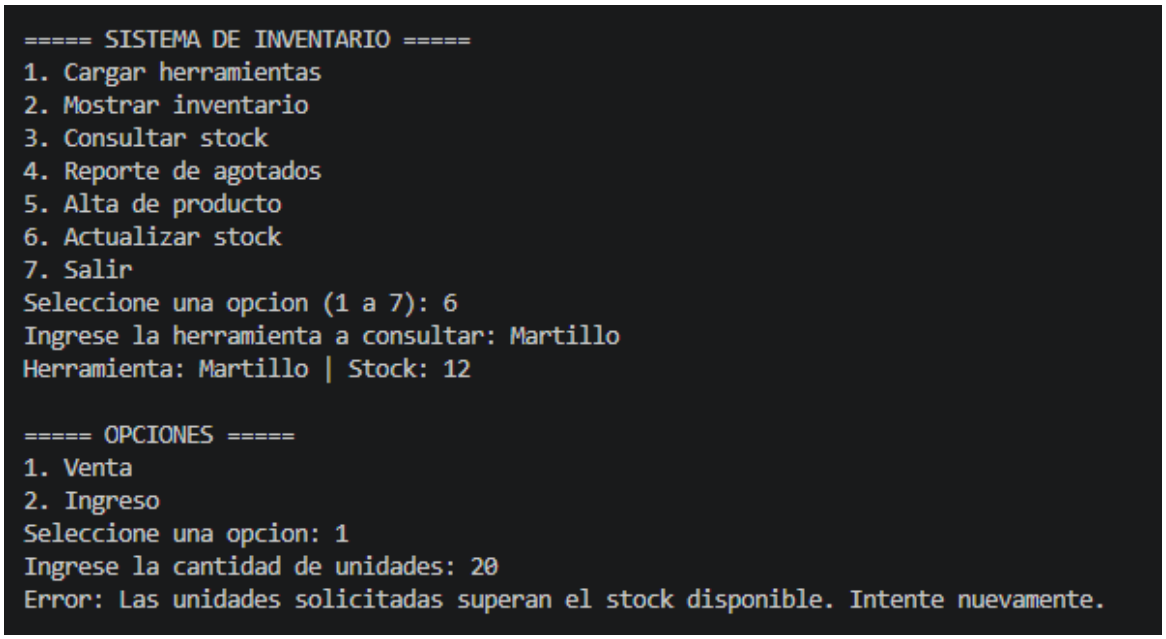
El resultado permitió verificar el funcionamiento del ingreso de mercadería y comprobar que la modificación se realizó únicamente sobre la herramienta seleccionada, sin afectar las cantidades de los demás productos.

Prueba 6. Validación de stock insuficiente

Intenté registrar una venta de 20 unidades cuando solamente había 12 disponibles.

El programa detectó que la cantidad solicitada superaba el stock existente y mostró un mensaje de error.

Posteriormente, consulté nuevamente el inventario y comprobé que las 12 unidades originales se habían conservado, evitando una modificación incorrecta.

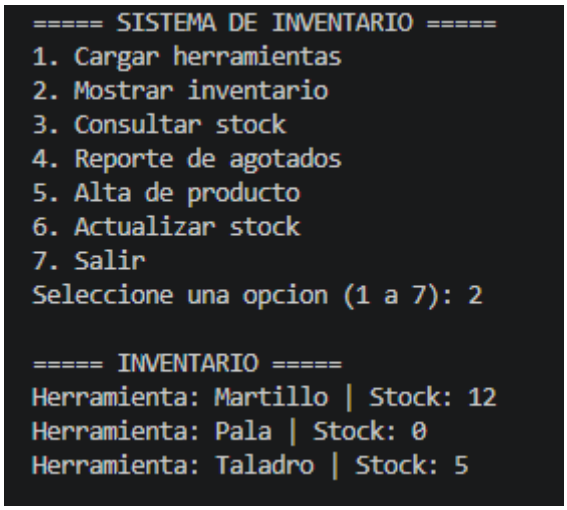


- **Prueba de validación por stock insuficiente.**

En esta prueba intenté registrar una venta de 20 unidades de la herramienta Martillo, cuando solamente había 12 unidades disponibles en el inventario.

El programa detectó que la cantidad solicitada superaba el stock existente y mostró un mensaje de error, impidiendo que se realizara la venta.

Esta prueba permitió comprobar el funcionamiento de las validaciones implementadas mediante estructuras condicionales y raise ValueError, evitando que una operación incorrecta genere cantidades negativas en el inventario.



- **Verificación del inventario después de una venta rechazada.**

Después de intentar realizar una venta superior al stock disponible, seleccioné la opción 2 para consultar nuevamente el inventario.

Al visualizar los resultados, comprobé que Martillo conservaba sus 12 unidades y que las cantidades de las demás herramientas tampoco habían cambiado.

Esta prueba permitió confirmar que las validaciones se ejecutan antes de modificar los datos, evitando que una operación incorrecta afecte al inventario.

Prueba 7. Manejo de entradas incorrectas

Realicé diferentes pruebas ingresando movimientos inválidos, cantidades negativas, nombres vacíos y valores no numéricos.

En cada caso, el programa mostró el mensaje correspondiente y evitó realizar operaciones incorrectas.

Estas pruebas permitieron comprobar la utilidad de las validaciones y del manejo de y

Resultados obtenidos

Las pruebas realizadas permitieron comprobar que las principales funciones del sistema respondieron de acuerdo con lo esperado.

El programa logró registrar herramientas, consultar cantidades, identificar productos agotados y actualizar el stock mediante ventas e ingresos de mercadería.

También pude verificar que las validaciones resultaron importantes para evitar errores y conservar la información del inventario ante operaciones inválidas.

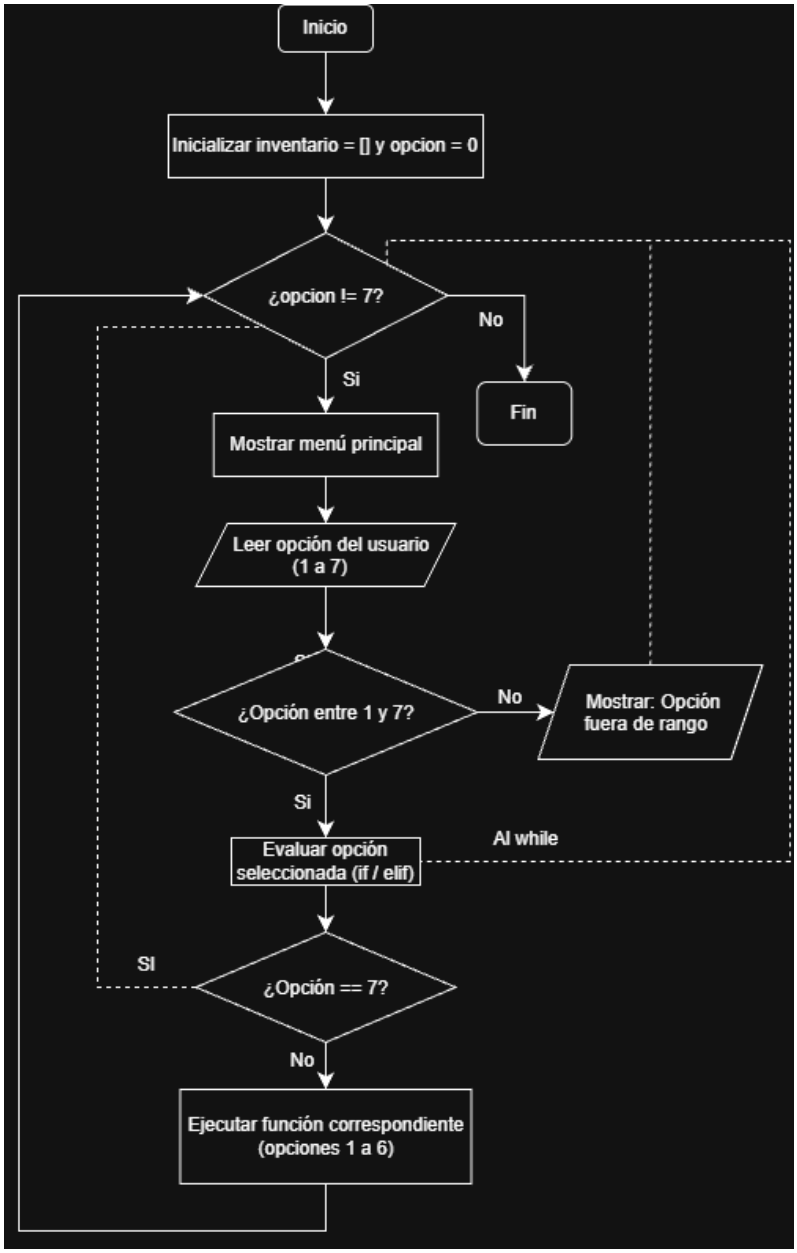
Durante este proceso fue necesario revisar y corregir algunas partes del código, lo que me permitió comprender mejor el funcionamiento de las funciones, los ciclos y el manejo de excepciones en Python.

- **Diagrama de flujo general del sistema de inventario.**

En este diagrama se representa el funcionamiento general del sistema de inventario. El programa comienza inicializando una lista vacía donde se almacenarán las herramientas y luego muestra un menú principal con siete opciones disponibles.

A partir de la opción seleccionada por el usuario, el sistema verifica que el valor ingresado sea válido y ejecuta la función correspondiente. Si se ingresa una opción incorrecta, se informa el error y se vuelve al menú principal.

Este proceso se repite mediante un ciclo while, permitiendo realizar distintas operaciones sin tener que reiniciar el programa. La ejecución finaliza únicamente cuando el usuario selecciona la opción 7, correspondiente a salir del sistema.

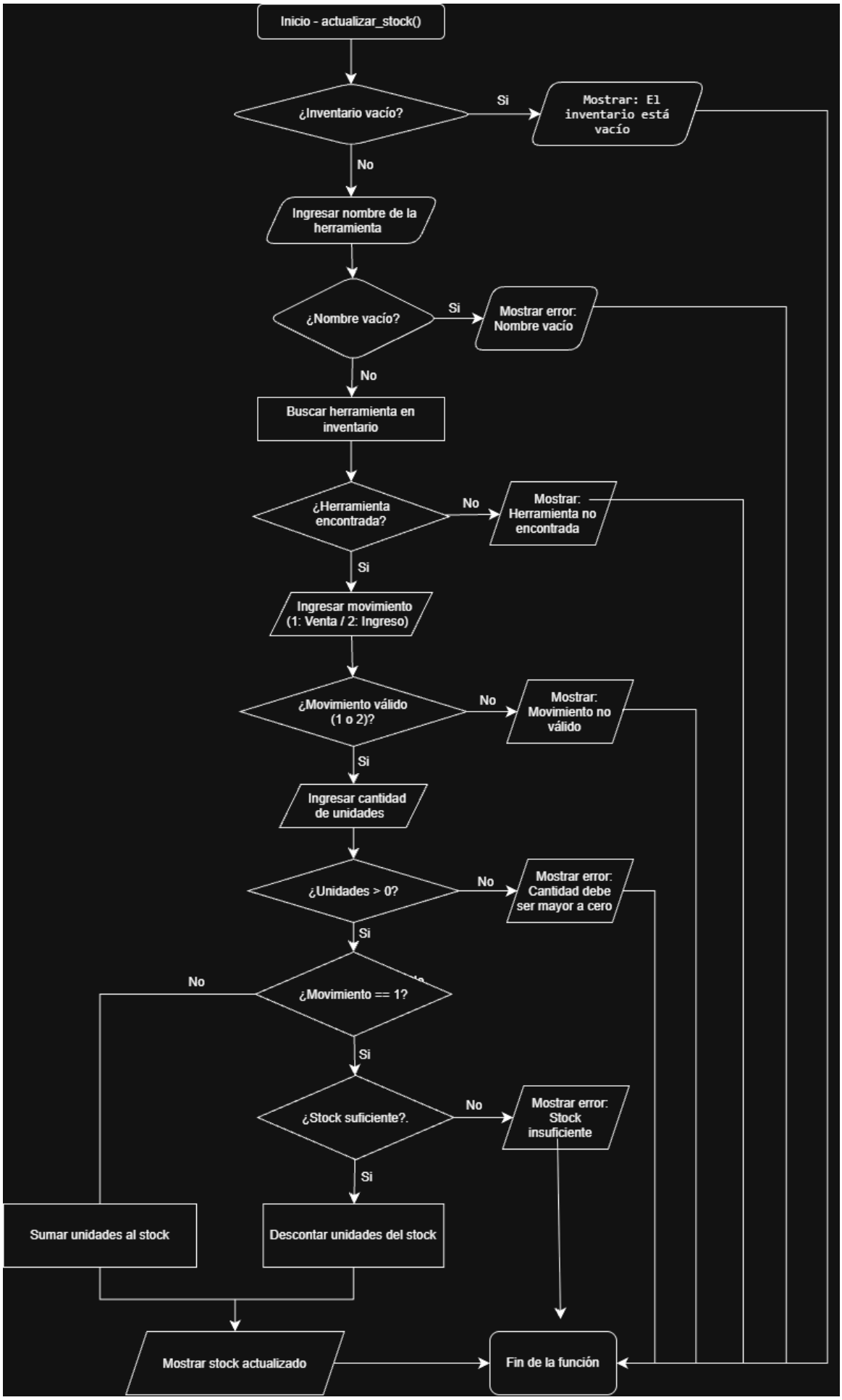


- **Diagrama de flujo de la función actualizar_stock().**

En este diagrama se representa el funcionamiento de la actualización del inventario. Primero, el programa verifica que existan herramientas cargadas y que el nombre ingresado sea válido. Luego, permite seleccionar entre realizar una venta o registrar un ingreso de mercadería.

En el caso de las ventas, se comprueba que haya stock suficiente antes de descontar las unidades. Si se trata de un ingreso, las unidades se suman al stock existente. También se incluyen las validaciones necesarias para evitar cantidades incorrectas y operaciones que puedan generar inconsistencias en el inventario.

De esta manera, se busca mantener un control adecuado de las existencias y evitar modificaciones incorrectas.


Inicio - actualizar_stock()
¿Inventario vacío?
Si
Mostrar: El inventario está vacío
No
Ingresar nombre de la herramienta
¿Nombre vacío?
Si
Mostrar error: Nombre vacío
No
Buscar herramienta en inventario
¿Herramienta encontrada?
No
Mostrar: Herramienta no encontrada
Si
Ingresar movimiento (1: Venta / 2: Ingreso)
¿Movimiento válido (1 o 2)?
No
Mostrar: Movimiento no válido
Si
Ingresar cantidad de unidades
¿Unidades > 0?
No
Mostrar error: Cantidad debe ser mayor a cero
Si
¿Movimiento == 1?
No
Si
¿Stock suficiente?.
No
Mostrar error: Stock insuficiente
Si
Sumar unidades al stock
Descontar unidades del stock
Mostrar stock actualizado
Fin de la función

Conclusión

A través de este Trabajo Práctico Integrador pude poner en práctica los conocimientos adquiridos durante la cursada de Programación I, desarrollando un sistema de inventario para una ferretería utilizando Python.

Durante el desarrollo utilicé diferentes herramientas de programación, como funciones, listas, diccionarios, estructuras condicionales, ciclos repetitivos y manejo de excepciones. Estos conceptos me permitieron organizar el programa y lograr que las distintas opciones del menú funcionaran de manera conjunta.

Una de las partes que más trabajo me llevó fue implementar las validaciones y la actualización del stock, especialmente al momento de controlar las cantidades disponibles y evitar operaciones incorrectas. Para resolver estas dificultades tuve que revisar el código, identificar errores y realizar distintas pruebas hasta conseguir los resultados esperados.

También comprendí la importancia de dividir un programa en funciones, ya que esto facilita su organización, permite detectar problemas con mayor facilidad y hace que el código sea más sencillo de mantener.

Como resultado final, logré desarrollar un sistema que permite registrar herramientas, consultar cantidades, identificar productos agotados y realizar operaciones de venta e ingreso de mercadería, manteniendo un control sobre los datos ingresados.

Considero que este trabajo fue una experiencia importante para mi aprendizaje, porque me permitió pasar de realizar ejercicios individuales a desarrollar un programa más completo, relacionando diferentes conceptos y comprendiendo mejor cómo aplicarlos para resolver un problema concreto.